

SQL ASSIGNMENT: DATA EXTRACTION AND ANALYSIS

BY: AYODELE JOSEPH OLAWOLE

TO: DECODELABS

1. Introduction

This project uses SQL queries to explore and analyze an Orders dataset. The aim is to extract useful insights from the data using basic SQL operations such as SELECT, WHERE, ORDER BY, GROUP BY, and aggregate functions like COUNT, SUM, and AVG.

The dataset contains information about customer orders including product details, quantity, payment method, order status, and total price.

2. Dataset Description

The dataset used for this analysis is the Orders dataset. It contains the following columns:

- OrderID
- Date
- CustomerID
- Product
- Quantity
- UnitPrice
- ShippingAddress
- PaymentMethod
- OrderStatus
- TrackingNumber
- ItemsInCart
- CouponCode
- ReferralSource
- TotalPrice

3. Tools Used

- SQL Server Management Studio (SSMS)

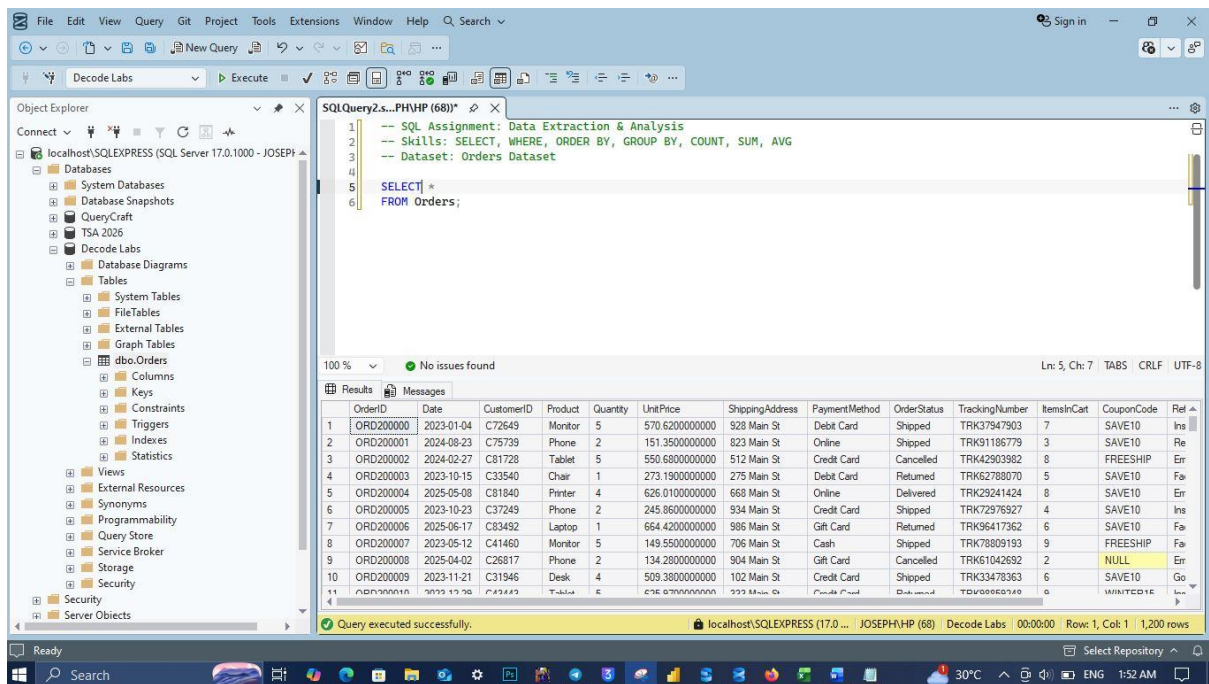
- Microsoft SQL Server
- Excel (for initial data preparation and import)

4. SQL Queries and Analysis

Query 1: View all records

SELECT *

FROM Orders;



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the 'dbo.Orders' table selected. The right pane shows the SQL Query Editor with the following query:

```

1  -- SQL Assignment: Data Extraction & Analysis
2  -- Skills: SELECT, WHERE, ORDER BY, GROUP BY, COUNT, SUM, AVG
3  -- Dataset: Orders Dataset
4
5  SELECT *
6  FROM Orders;

```

The query results are displayed in a table with the following columns: OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, and Ref. The table contains 10 rows of data.

OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	Ref
ORD200000	2023-01-04	C72649	Monitor	5	570.6200000000	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Ins
ORD200001	2024-08-23	C75739	Phone	2	151.3500000000	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Re
ORD200002	2024-02-27	C81728	Tablet	5	550.6800000000	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Err
ORD200003	2023-10-15	C33540	Chair	1	273.1900000000	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	Fa
ORD200004	2025-05-08	C81840	Printer	4	626.0100000000	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Err
ORD200005	2023-10-23	C37249	Phone	2	245.8600000000	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Ins
ORD200006	2025-06-17	C83492	Laptop	1	664.4200000000	986 Main St	Gift Card	Returned	TRK96417362	6	SAVE10	Fa
ORD200007	2023-05-12	C41460	Monitor	5	149.5500000000	706 Main St	Cash	Shipped	TRK78809193	9	FREESHIP	Fa
ORD200008	2025-04-02	C26817	Phone	2	134.2800000000	904 Main St	Gift Card	Cancelled	TRK61042692	2	NULL	Err
ORD200009	2023-11-21	C31946	Desk	4	509.3800000000	102 Main St	Credit Card	Shipped	TRK33478363	6	SAVE10	Go

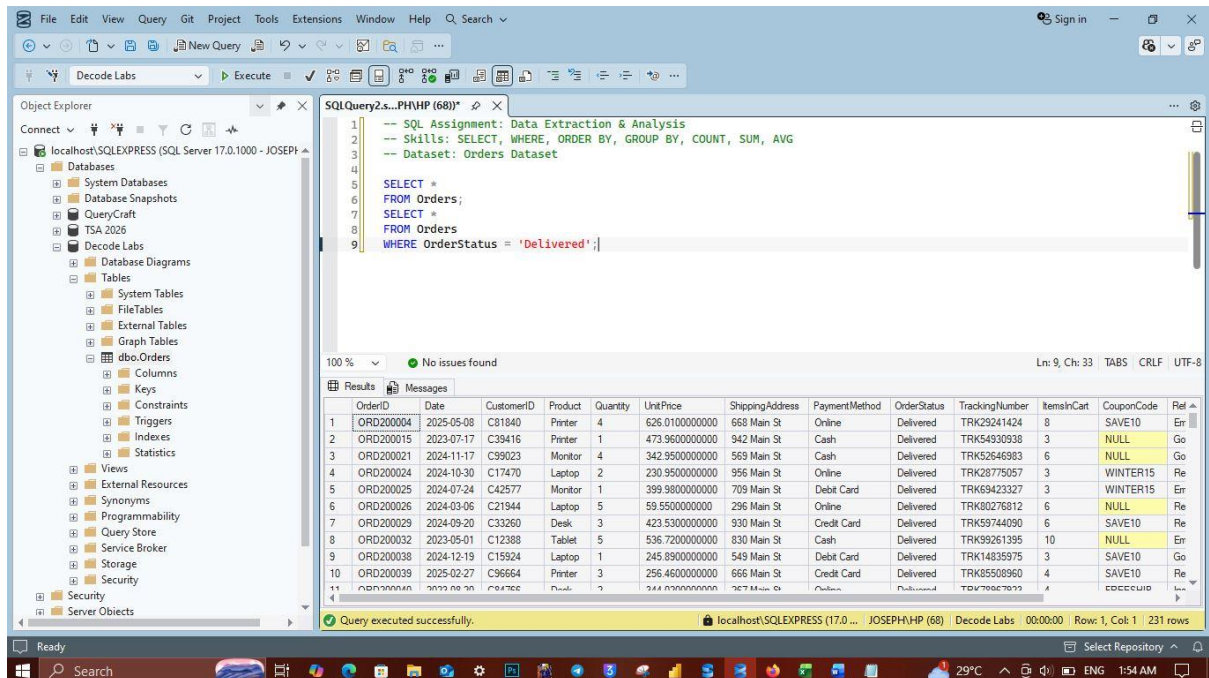
The status bar at the bottom indicates: Query executed successfully. Localhost\SQLEXPRESS (17.0 ... JOSEPH\HP (68) Decode Labs 00:00:00 Row: 1, Col: 1 1,200 rows

Explanation:

This query displays all records in the Orders table to understand the structure and contents of the dataset.

Query 2: Filter delivered orders

```
SELECT *  
  
FROM Orders  
  
WHERE OrderStatus = 'Delivered';
```



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with the 'dbo.Orders' table selected. The central pane shows the SQL query editor with the following query:

```
-- SQL Assignment: Data Extraction & Analysis  
-- Skills: SELECT, WHERE, ORDER BY, GROUP BY, COUNT, SUM, AVG  
-- Dataset: Orders Dataset  
  
SELECT *  
FROM Orders;  
SELECT *  
FROM Orders  
WHERE OrderStatus = 'Delivered';
```

The bottom pane shows the 'Results' tab with a table of 11 rows. The table columns are: OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, and Ref. The 'OrderStatus' column is filtered to show only 'Delivered' orders.

OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	Ref
ORD200004	2025-05-08	C81840	Printer	4	626.0100000000	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Err
ORD200015	2023-07-17	C39416	Printer	1	473.9600000000	942 Main St	Cash	Delivered	TRK54930938	3	NULL	Go
ORD200021	2024-11-17	C99023	Monitor	4	342.9500000000	569 Main St	Cash	Delivered	TRK52646983	6	NULL	Go
ORD200024	2024-10-30	C17470	Laptop	2	230.9500000000	956 Main St	Online	Delivered	TRK28775057	3	WINTER15	Re
ORD200025	2024-07-24	C42577	Monitor	1	399.9800000000	709 Main St	Debit Card	Delivered	TRK69423327	3	WINTER15	Err
ORD200026	2024-03-06	C21944	Laptop	5	59.5500000000	296 Main St	Online	Delivered	TRK80276812	6	NULL	Re
ORD200029	2024-09-20	C33260	Desk	3	423.5300000000	930 Main St	Credit Card	Delivered	TRK59744090	6	SAVE10	Re
ORD200032	2023-05-01	C12388	Tablet	5	536.7200000000	830 Main St	Cash	Delivered	TRK99261395	10	NULL	Err
ORD200038	2024-12-19	C15924	Laptop	1	245.8900000000	549 Main St	Debit Card	Delivered	TRK14835975	3	SAVE10	Go
ORD200039	2025-02-27	C96664	Printer	3	256.4600000000	666 Main St	Credit Card	Delivered	TRK85508960	4	SAVE10	Re
ORD200040	2025-09-20	C94766	Desk	2	244.0000000000	267 Main St	Online	Delivered	TRK770667072	4	EDCCCU10	Go

Explanation:

This query filters the dataset to show only orders that have been successfully delivered.

Query 3: Sort orders by total price

```
SELECT *
```

```
FROM Orders
```

```
ORDER BY TotalPrice DESC;
```

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'dbo.Orders' table selected. The right pane shows the SQL Query window with the following query:

```
-- SQL Assignment: Data Extraction & Analysis
-- Skills: SELECT, WHERE, ORDER BY, GROUP BY, COUNT, SUM, AVG
-- Dataset: Orders Dataset

SELECT *
FROM Orders;
SELECT *
FROM Orders
WHERE OrderStatus = 'Delivered';
SELECT *
FROM Orders
ORDER BY TotalPrice DESC;
```

The query results are displayed in a table with the following columns: OrderID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, and TotalPrice. The results are sorted by TotalPrice in descending order.

OrderID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	Tablet	5	691.2800000000	183 Main St	Online	Delivered	TRK75899752	10	SAVE10	Email	3456.4000000000
2	Monitor	5	678.1900000000	766 Main St	Online	Returned	TRK32496970	8	NULL	Facebook	3390.9500000000
3	Laptop	5	678.1600000000	463 Main St	Gift Card	Delivered	TRK38229104	7	WINTER15	Facebook	3390.8000000000
4	Chair	5	676.9800000000	893 Main St	Cash	Cancelled	TRK17254691	5	NULL	Facebook	3384.9000000000
5	Tablet	5	674.0400000000	546 Main St	Online	Cancelled	TRK89401624	7	SAVE10	Google	3370.2000000000
6	Printer	5	670.7500000000	848 Main St	Gift Card	Shipped	TRK34392124	8	FREESHIP	Instagram	3353.7500000000
7	Laptop	5	670.4800000000	273 Main St	Gift Card	Returned	TRK98353867	5	SAVE10	Facebook	3352.4000000000
8	Printer	5	666.8000000000	488 Main St	Debit Card	Delivered	TRK79504329	7	SAVE10	Referral	3334.0000000000
9	Phone	5	664.5100000000	136 Main St	Debit Card	Pending	TRK18129706	8	SAVE10	Email	3322.5500000000
10	Laptop	5	662.7800000000	214 Main St	Debit Card	Shipped	TRK14011732	9	SAVE10	Instagram	3313.9000000000
11	Desktop	5	660.0000000000	107 Main St	Credit Card	Delivered	TRK77719731	6	SAVE10	Facebook	3300.7500000000

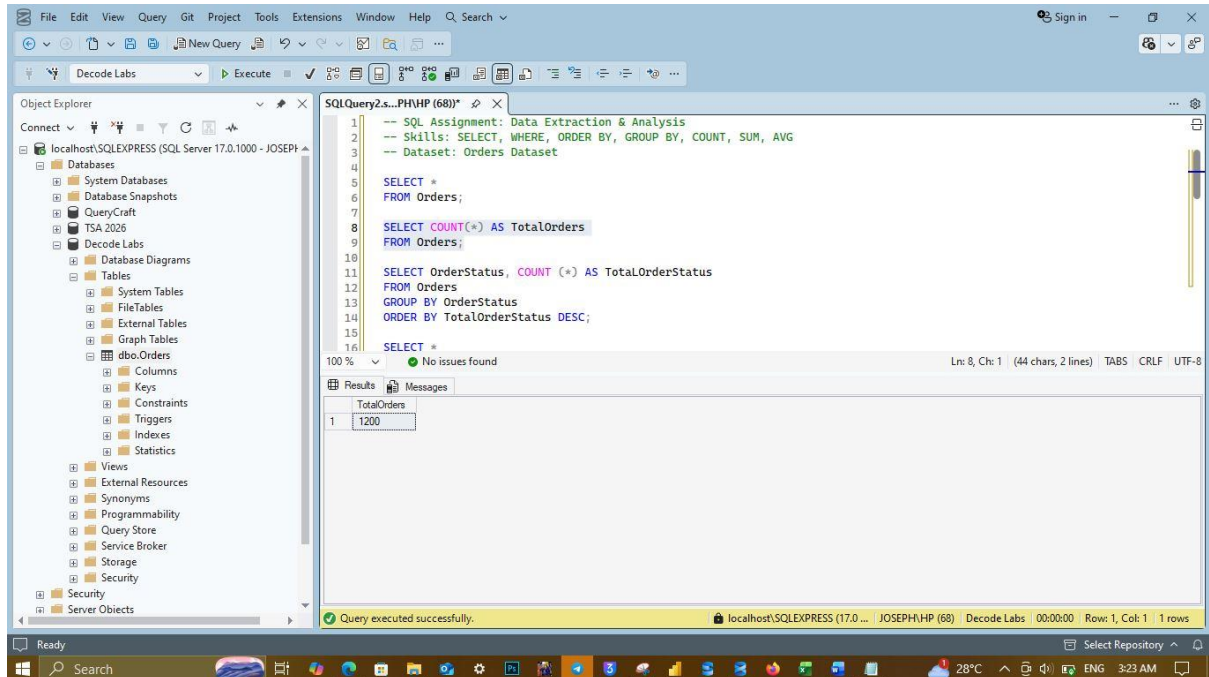
The status bar at the bottom indicates that the query was executed successfully, showing 1,200 rows.

Explanation:

This query sorts all orders in descending order based on total price to identify the highest value orders.

Query 4: Total Orders

```
SELECT COUNT(*) AS TotalOrders  
FROM Orders;
```

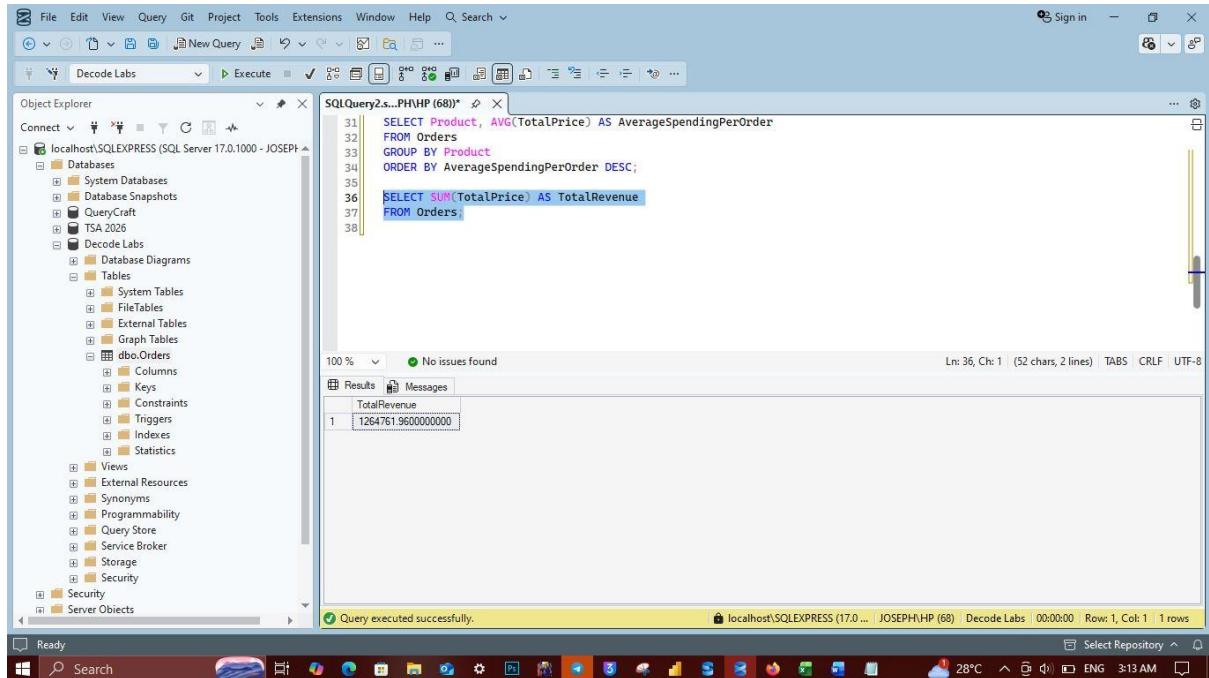


Explanation:

This query returns the total number of orders recorded in the dataset. It gives a general overview of dataset size and helps in understanding the scale of transactions behind the revenue and analysis.

Query 5: Total Revenue

```
SELECT SUM(TotalPrice) AS TotalRevenue  
FROM Orders;
```

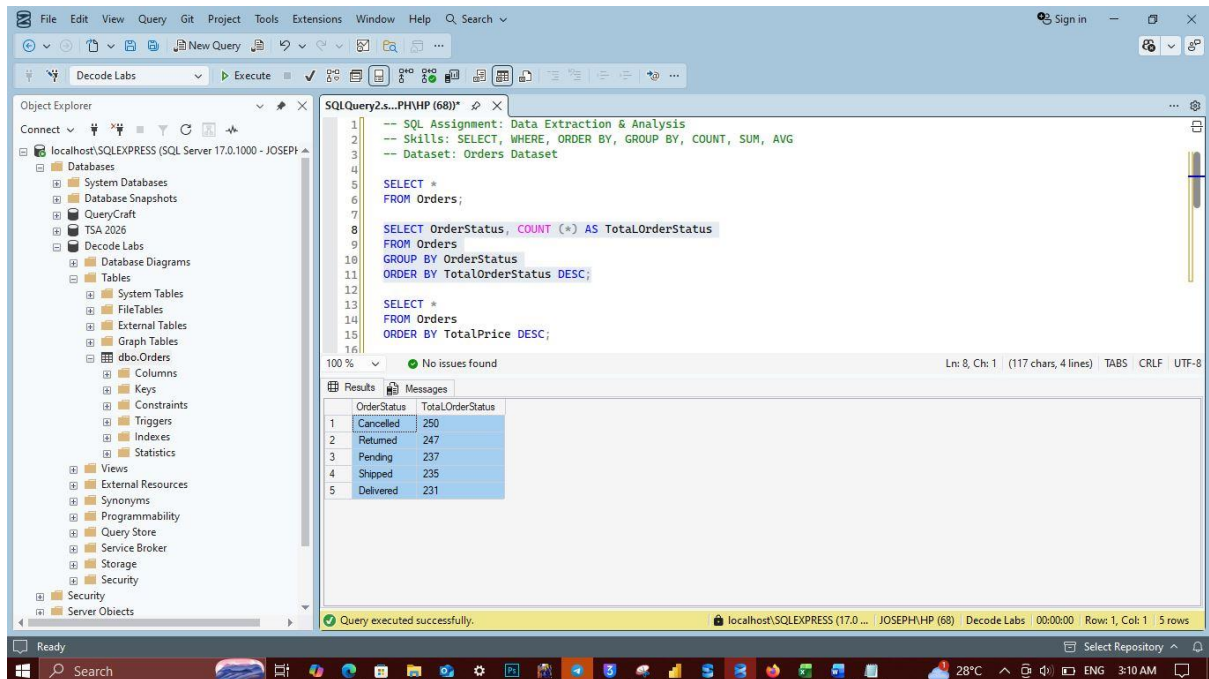


Explanation:

This query calculates the total revenue generated from all orders in the dataset by summing up the values in the TotalPrice column. It provides a single overall figure representing the total sales performance.

Query 6: Order Status Distribution

```
SELECT OrderStatus, COUNT(*) AS TotalOrderStatus
FROM Orders
GROUP BY OrderStatus
ORDER BY TotalOrderStatus DESC;
```



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'dbo.Orders' table selected. The right pane shows the SQL Query window with the following query:

```
-- SQL Assignment: Data Extraction & Analysis
-- Skills: SELECT, WHERE, ORDER BY, GROUP BY, COUNT, SUM, AVG
-- Dataset: Orders Dataset

SELECT *
FROM Orders;

SELECT OrderStatus, COUNT(*) AS TotalOrderStatus
FROM Orders
GROUP BY OrderStatus
ORDER BY TotalOrderStatus DESC;

SELECT *
FROM Orders
ORDER BY TotalPrice DESC;
```

The query results are displayed in the Results pane, showing a table with two columns: OrderStatus and TotalOrderStatus. The results are ordered by TotalOrderStatus in descending order.

OrderStatus	TotalOrderStatus
Cancelled	250
Returned	247
Pending	237
Shipped	235
Delivered	231

The status bar at the bottom indicates that the query was executed successfully, showing the number of rows (5) and columns (2).

Explanation:

This query groups all orders based on their order status and counts the number of orders in each category. It helps to understand the distribution of order outcomes such as Delivered, Pending, Shipped, Cancelled, or Returned.

Query 7: Orders by Payment Method

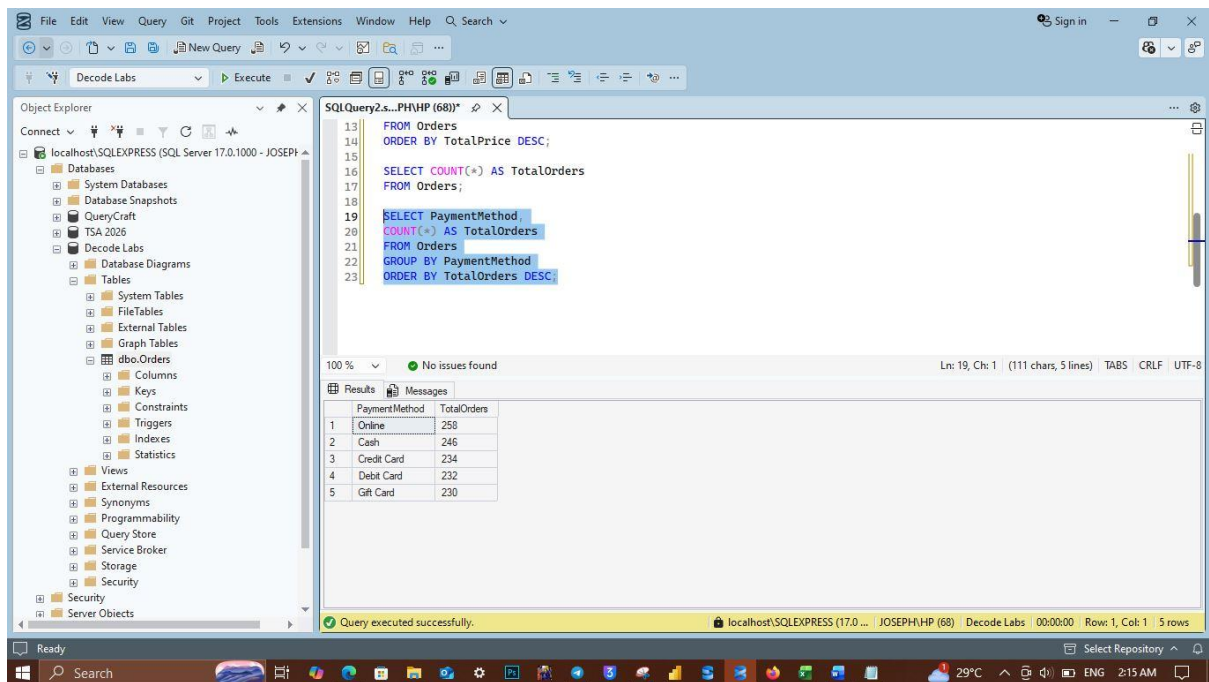
```
SELECT PaymentMethod,
```

```
COUNT(*) AS TotalOrders
```

```
FROM Orders
```

```
GROUP BY PaymentMethod
```

```
ORDER BY TotalOrders DESC;
```



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'localhost\SQLEXPRESS (SQL Server 17.0.1000 - JOSEPH)'. The central pane shows a query window with the following SQL code:

```
13 FROM Orders
14 ORDER BY TotalPrice DESC;
15
16 SELECT COUNT(*) AS TotalOrders
17 FROM Orders;
18
19 SELECT PaymentMethod,
20 COUNT(*) AS TotalOrders
21 FROM Orders
22 GROUP BY PaymentMethod
23 ORDER BY TotalOrders DESC;
```

The query results are displayed in a table with two columns: 'PaymentMethod' and 'TotalOrders'. The results are ordered by 'TotalOrders' in descending order.

PaymentMethod	TotalOrders
Online	258
Cash	246
Credit Card	234
Debit Card	232
Gift Card	230

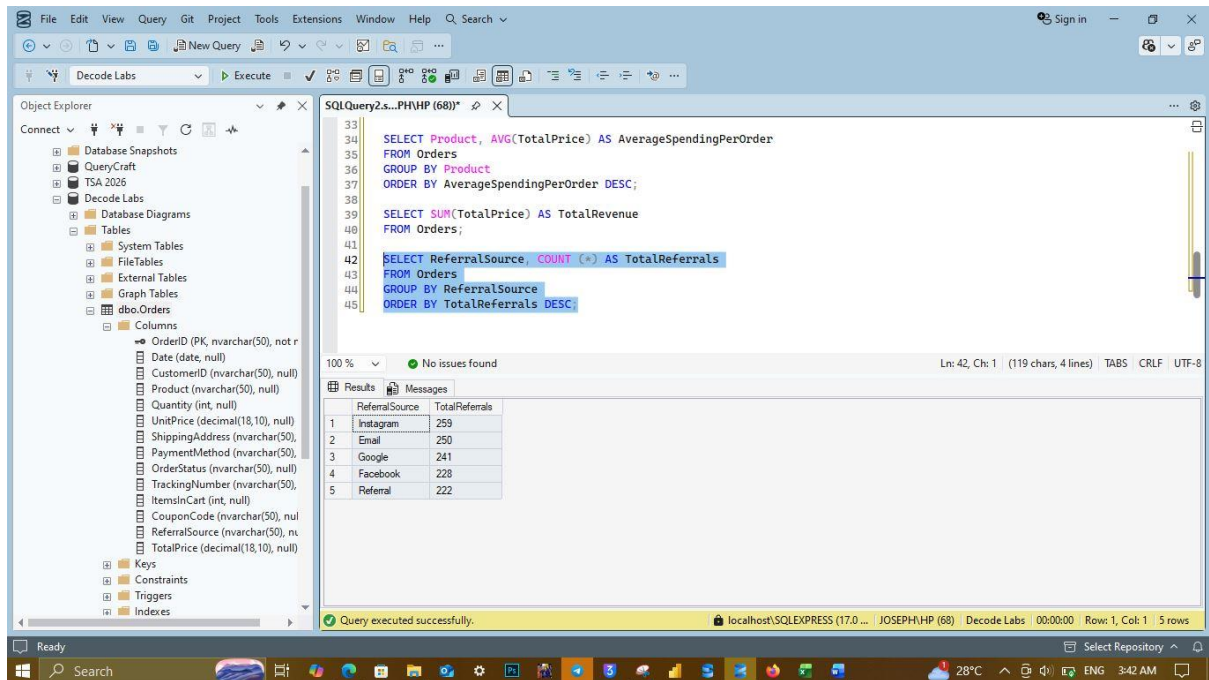
The status bar at the bottom indicates 'Query executed successfully.' and 'localhost\SQLEXPRESS (17.0 ... JOSEPH:HP (68) Decode Labs 00:00:00 Rows: 1, Col: 1 5 rows'.

Explanation:

This query shows the number of orders made using each payment method, helping to identify the most commonly used payment option.

Query 8: Orders by Referral Source

```
SELECT ReferralSource, COUNT(*) AS TotalReferrals
FROM Orders
GROUP BY ReferralSource
ORDER BY TotalReferrals DESC;
```



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure, including tables, keys, constraints, triggers, and indexes. The central pane shows a SQL query window with the following code:

```
33 SELECT Product, AVG(TotalPrice) AS AverageSpendingPerOrder
34 FROM Orders
35 GROUP BY Product
36 ORDER BY AverageSpendingPerOrder DESC;
37
38 SELECT SUM(TotalPrice) AS TotalRevenue
39 FROM Orders;
40
41
42 SELECT ReferralSource, COUNT (*) AS TotalReferrals
43 FROM Orders
44 GROUP BY ReferralSource
45 ORDER BY TotalReferrals DESC;
```

The query results are displayed in the bottom pane, showing a table with two columns: ReferralSource and TotalReferrals. The results are as follows:

ReferralSource	TotalReferrals
Instagram	259
Email	250
Google	241
Facebook	228
Referral	222

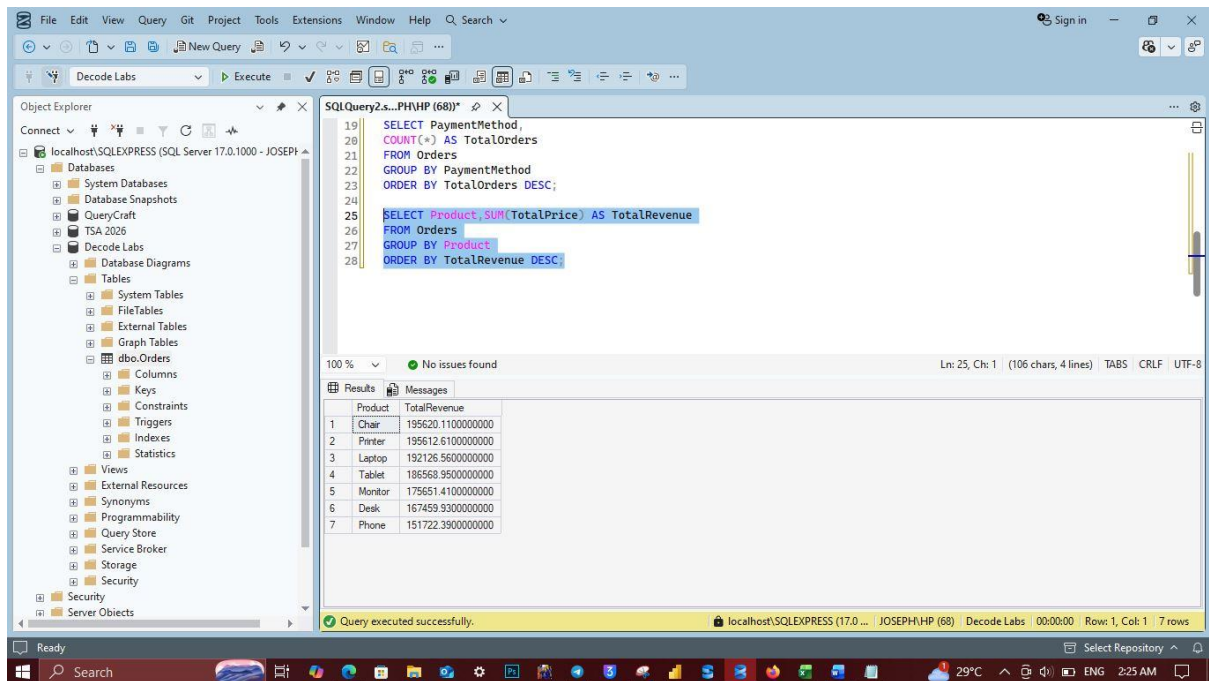
The status bar at the bottom indicates that the query was executed successfully on the local host SQL EXPRESS (17.0) database.

Explanation:

This query groups orders based on their referral source and counts how many orders came from each channel. It helps to identify which platforms or sources bring in the most customers.

Query 9: Total revenue per product

```
SELECT Product, SUM(TotalPrice) AS TotalRevenue  
FROM Orders  
GROUP BY Product  
ORDER BY TotalRevenue DESC;
```



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'dbo.Orders' table selected. The right pane shows the SQL Query Editor with the following query:

```
19 SELECT PaymentMethod,  
20 COUNT(*) AS TotalOrders  
21 FROM Orders  
22 GROUP BY PaymentMethod  
23 ORDER BY TotalOrders DESC;  
24  
25 SELECT Product, SUM(TotalPrice) AS TotalRevenue  
26 FROM Orders  
27 GROUP BY Product  
28 ORDER BY TotalRevenue DESC;
```

The query results are displayed in the Results pane, showing a table with two columns: Product and TotalRevenue. The results are ordered by TotalRevenue in descending order.

Product	TotalRevenue
Chair	195620.11000000000
Printer	195612.61000000000
Laptop	192126.56000000000
Tablet	186568.95000000000
Monitor	179651.41000000000
Desk	167459.93000000000
Phone	151722.39000000000

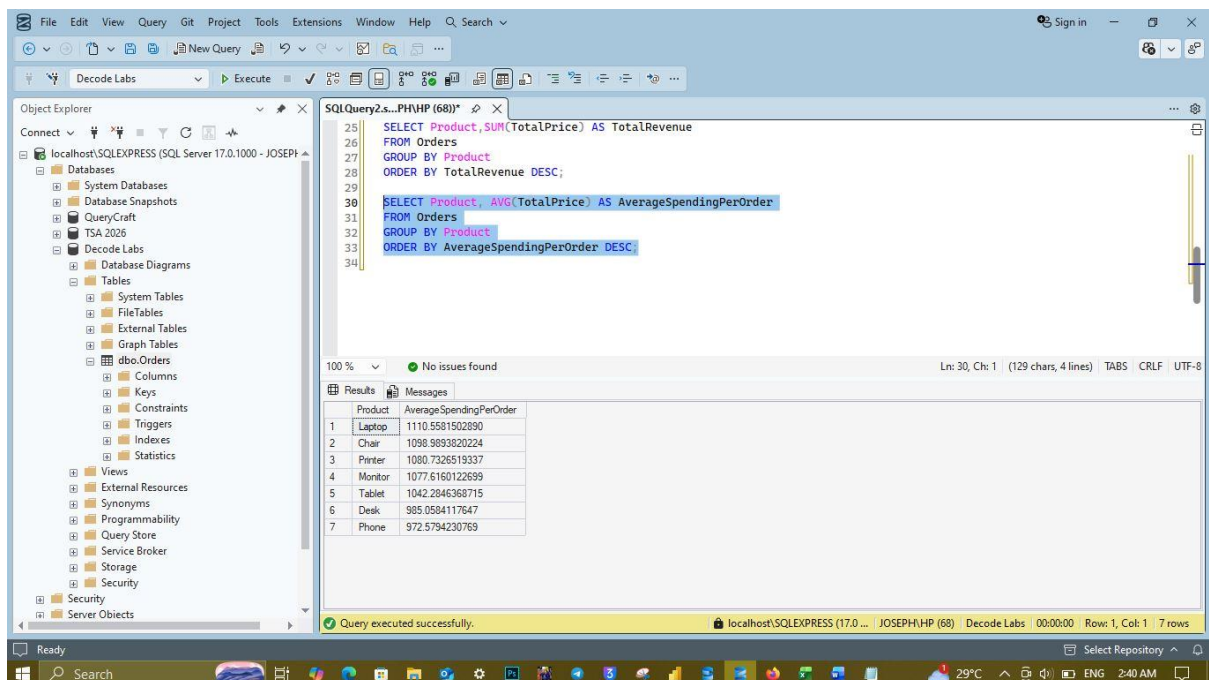
The status bar at the bottom indicates that the query was executed successfully.

Explanation:

This query calculates the total revenue generated by each product.

Query 10: Average spending per product

```
SELECT Product, AVG(TotalPrice) AS AverageSpendingPerOrder  
FROM Orders  
GROUP BY Product  
ORDER BY AverageSpendingPerOrder DESC;
```



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'dbo.Orders' table selected. The right pane shows the SQL Query window with the following query:

```
25 SELECT Product, SUM(TotalPrice) AS TotalRevenue  
26 FROM Orders  
27 GROUP BY Product  
28 ORDER BY TotalRevenue DESC;  
29  
30 SELECT Product, AVG(TotalPrice) AS AverageSpendingPerOrder  
31 FROM Orders  
32 GROUP BY Product  
33 ORDER BY AverageSpendingPerOrder DESC;  
34
```

The query results are displayed in the Results pane, showing a table with two columns: Product and AverageSpendingPerOrder. The results are ordered by AverageSpendingPerOrder in descending order.

Product	AverageSpendingPerOrder
Laptop	1110.5581502690
Chair	1098.9893820224
Printer	1080.7326519337
Monitor	1077.6160122699
Tablet	1042.2846368715
Desk	985.0584117647
Phone	972.5794230769

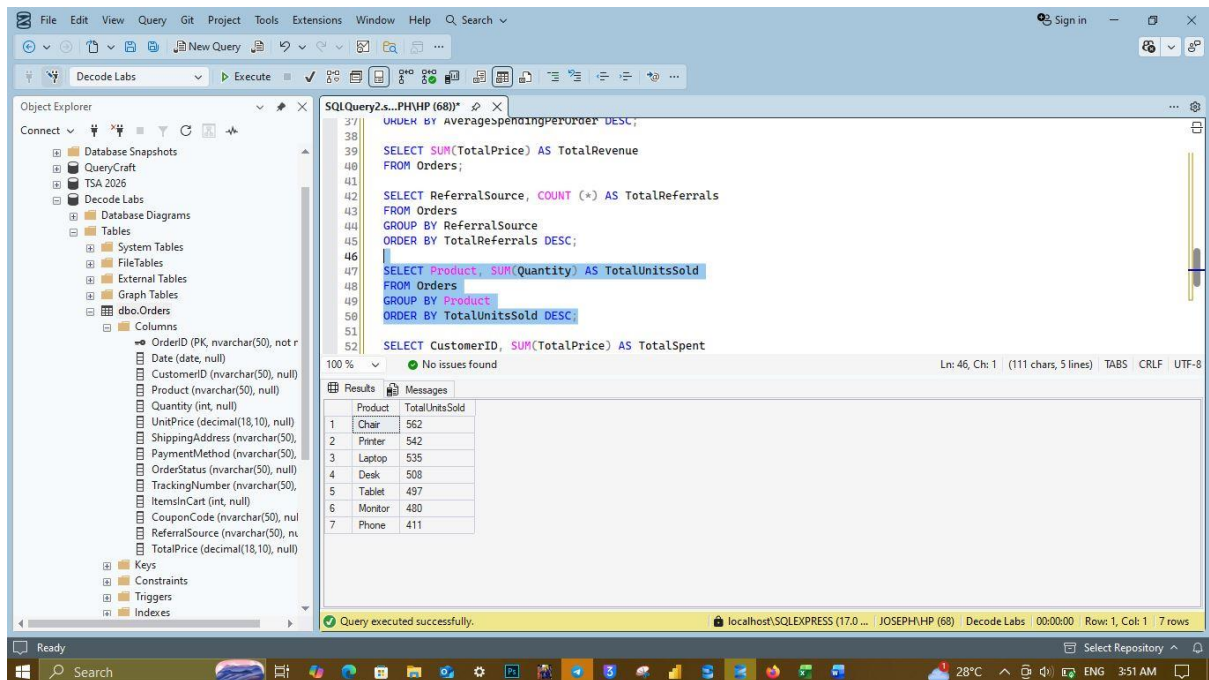
The status bar at the bottom indicates that the query was executed successfully.

Explanation:

This query calculates the average order value for each product to understand spending patterns.

Query 11: Total Units Sold per Product

```
SELECT Product, SUM(Quantity) AS TotalUnitsSold
FROM Orders
GROUP BY Product
ORDER BY TotalUnitsSold DESC;
```



The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with the 'Decode Labs' database selected. The right pane shows the 'SQL Query 2' window with the following query:

```
SELECT SUM(TotalPrice) AS TotalRevenue
FROM Orders;

SELECT ReferralSource, COUNT(*) AS TotalReferrals
FROM Orders
GROUP BY ReferralSource
ORDER BY TotalReferrals DESC;

SELECT Product, SUM(Quantity) AS TotalUnitsSold
FROM Orders
GROUP BY Product
ORDER BY TotalUnitsSold DESC;

SELECT CustomerID, SUM(TotalPrice) AS TotalSpent
```

The query results are displayed in the 'Results' pane, showing a table with two columns: 'Product' and 'TotalUnitsSold'. The results are ordered by 'TotalUnitsSold' in descending order.

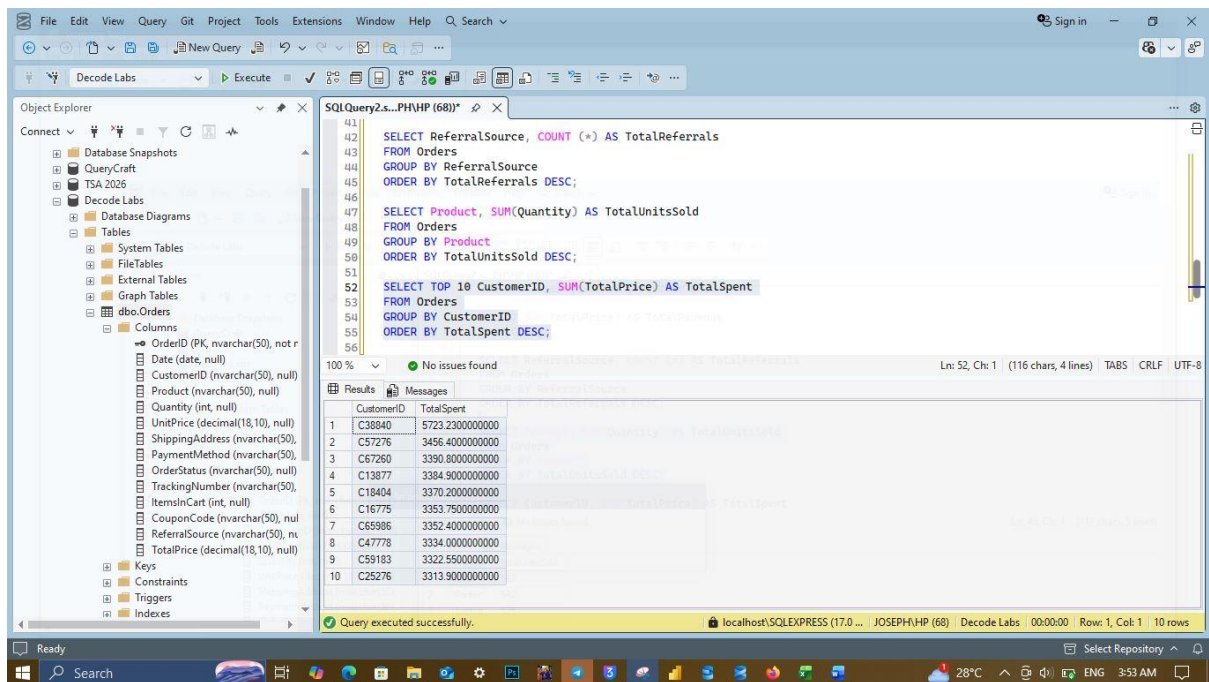
Product	TotalUnitsSold
Chair	562
Printer	542
Laptop	535
Desk	508
Tablet	497
Monitor	480
Phone	411

Explanation:

This query calculates the total number of units sold for each product by summing the Quantity column. It helps to identify which products are most frequently purchased regardless of price or revenue.

Query 12: Highest Spending Customers

```
SELECT TOP 10 CustomerID, SUM(TotalPrice) AS TotalSpent
FROM Orders
GROUP BY CustomerID
ORDER BY TotalSpent DESC;
```



The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the database structure, including tables like 'Orders'. The main pane shows a SQL query window with the following code:

```
41
42 SELECT ReferralSource, COUNT(*) AS TotalReferrals
43 FROM Orders
44 GROUP BY ReferralSource
45 ORDER BY TotalReferrals DESC;
46
47 SELECT Product, SUM(Quantity) AS TotalUnitsSold
48 FROM Orders
49 GROUP BY Product
50 ORDER BY TotalUnitsSold DESC;
51
52 SELECT TOP 10 CustomerID, SUM(TotalPrice) AS TotalSpent
53 FROM Orders
54 GROUP BY CustomerID
55 ORDER BY TotalSpent DESC;
56
```

Below the query, the Results tab shows the output of the third query, displaying the top 10 customers by total spending:

CustomerID	TotalSpent
C38840	5723.2300000000
C57276	3456.4000000000
C57260	3390.8000000000
C13877	3384.9000000000
C18404	3370.2000000000
C16775	3353.7500000000
C65986	3352.4000000000
C47778	3334.0000000000
C58183	3322.5500000000
C25276	3313.9000000000

The status bar at the bottom indicates the query was executed successfully on the 'localhost\SQLEXPRESS (17.0 ...)' instance, with 10 rows returned.

Explanation:

This query identifies the top 10 customers based on total spending by summing all their purchases. It helps to understand customer value and identify high-value buyers in the dataset.

5. Key Findings

From the SQL analysis performed on the Orders dataset, the following insights were observed:

- There are **1,200** orders in the dataset, providing enough data to observe patterns in revenue, payment methods, and order status.
- The total revenue generated from all orders is **1,264,761.96**, giving an overall view of the sales performance of the dataset.
- Revenue is relatively close across the top-performing products, with **Chair (195,620.11)** and **Printer (195,612.61)** leading by a very small margin. However, there is still a noticeable drop down to **Phone (151,722.39)**, showing that a few products perform slightly better than others.
- Average spending per order is highest for **Laptop (1110.56)** and **Chair (1098.99)**, while **Phone (972.58)** is the lowest. This shows that some products consistently generate higher-value purchases than others.
- **Chair** is the most purchased product with **562 units sold**, followed closely by **Printer (542 units)**, showing that these products have the highest demand in terms of quantity sold.
- Customer payment behaviour shows that **Online payment (258 orders)** is the most used method, closely followed by **Cash (246 orders)**. The differences are small, meaning customers use different payment options fairly evenly.
- Order status distribution is fairly spread out, with **Cancelled (250)** being the highest, followed by **Returned (247)**, **Pending (237)**, **Shipped (235)**, and **Delivered (231)**. This shows that order outcomes are distributed rather than dominated by completed deliveries.
- Customer **C38840** is the highest-spending customer with a total spend of **5,723.23**, significantly higher than other customers, indicating a strong outlier in customer value.

6. Conclusion

This project used basic SQL queries to explore and analyse an Orders dataset. By applying SELECT, WHERE, ORDER BY, GROUP BY, and aggregate functions, the data was broken down into useful insights about sales performance, customer behaviour, and order patterns.

The analysis shows that revenue is fairly distributed across products, with only small differences between top performers. Customer payment methods are also evenly spread, and order statuses show a balanced distribution across different outcomes. These results demonstrate how SQL can be used to quickly summarise and understand real-world business data.